

User Manual

for S32K1_S32M24X LINTRCV_43_AE Driver

Document Number: UM2LINTRCV_43_AEASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	3
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	6
3.4 Deviations from Requirements	7
3.5 Driver Limitations	9
3.6 Driver usage and configuration tips	9
3.6.1 How to configure the SPI sequence via AE module	9
3.6.2 Wakeup notification handling	13
3.6.3 Approach about examples of LINTRCV	18
3.7 Runtime errors	19
3.8 Symbolic Names Disclaimer	19
3.9 Usage in non AUTOSAR context	19
4 Configuration Parameters	21
4.1 LinTrcv	21
4.1.1 Module LinTrcv	22
4.1.2 Container LinTrcvGeneral	23
4.1.3 Parameter LinTrcvMultiPartitionSupport	23
4.1.4 Parameter LinTrcvDevErrorDetect	24
4.1.5 Parameter LinTrcvIndex	24
4.1.6 Parameter LinTrcvTimerType	24
4.1.7 Parameter LinTrcvVersionInfoApi	25
4.1.8 Parameter LinTrcvWaitTime	25
4.1.9 Parameter LinTrcvWakeUpSupport	26
4.1.10 Reference LinTrcvEcucPartitionRef	26
4.1.11 Container LinTrcvChannel	27
4.1.12 Parameter LinTrcvChannelId	27
4.1.13 Parameter LinTrcvHwChannel	27
4.1.14 Parameter LinTrcvChannelUsed	28
4.1.15 Parameter LinTrcvWakeupByBusUsed	28
4.1.16 Reference LinTrcvWakeupSourceRef	29
4.1.17 Reference LinTrcvChannelEcucPartitionRef	29
4.1.18 Reference LinTrcvIcuChannelRef	29
4.1.19 Container LinTrcvAccess	30
4.1.20 Container CommonPublishedInformation	30
4.1.21 Parameter ArReleaseMajorVersion	30
4.1.22 Parameter ArReleaseMinorVersion	31
4.1.23 Parameter ArReleaseRevisionVersion	31
4.1.24 Parameter ModuleId	32
4.1.25 Parameter SwMajorVersion	32
4.1.26 Parameter SwMinorVersion	32
4.1.27 Parameter SwPatchVersion	33
4.1.28 Parameter VendorApiInfix	33
4.1.29 Parameter VendorId	34

5 Module Index	35
5.1 Software Specification	35
6 Module Documentation	36
6.1 AELINPHY_LINTRCV_IP	36
6.1.1 Detailed Description	36
6.1.2 Function Reference	36
6.1.2.1 AeLinPhy_LinTrcv_Ip_Init()	36
6.1.2.2 AeLinPhy_LinTrcv_Ip_SetMode()	36
6.1.2.3 AeLinPhy_LinTrcv_Ip_GetMode()	36
6.1.2.4 AeLinPhy_LinTrcv_Ip_SetWakeupMode()	37
6.1.2.5 AeLinPhy_LinTrcv_Ip_CheckWakeupFlag()	37
6.2 AeLinPhy LinTrcv IPL	38
6.2.1 Detailed Description	38
6.2.2 Data Structure Documentation	38
6.2.2.1 struct AeLinPhy_LinTrcv_Ip_UserConfigType	38
6.2.3 Enum Reference	38
6.2.3.1 AeLinPhy_LinTrcv_Ip_TransceiverModeType	38
6.2.3.2 AeLinPhy_LinTrcv_Ip_StatusType	39
6.2.3.3 AeLinPhy_LinTrcv_Ip_TrcvWakeupModeType	39
6.3 LINTRCV Driver	40
6.3.1 Detailed Description	40
6.3.2 Macro Definition Documentation	40
6.3.2.1 LINTRCV_43_AE_NOT_ACTIVE	41
6.3.2.2 LINTRCV_43_AE_ACTIVE	41
6.3.2.3 LINTRCV_43_AE_SETOPMODE_ID	41
6.3.2.4 LINTRCV_43_AE_GETOPMODE_ID	41
6.3.2.5 LINTRCV_43_AE_GETBUSWUREASON_ID	41
6.3.2.6 LINTRCV_43_AE_CHECKWAKEUP_ID	42
6.3.2.7 LINTRCV_43_AE_SETWAKEUPMODE_ID	42
6.3.3 Function Reference	42
6.3.3.1 LinTrcv_43_AE_Init()	42
6.3.3.2 LinTrcv_43_AE_SetOpMode()	42
6.3.3.3 LinTrcv_43_AE_GetOpMode()	43
6.3.3.4 LinTrcv_43_AE_GetBusWuReason()	43
6.3.3.5 LinTrcv_43_AE_CheckWakeup()	44
6.3.3.6 LinTrcv_43_AE_SetWakeupMode()	44
6.4 LINTRCV Driver	46
6.4.1 Detailed Description	46
6.4.2 Data Structure Documentation	46
6.4.2.1 struct LinTrcv_43_AE_ChannelConfigType	46
6.4.2.2 struct LinTrcv_43_AE_ConfigType	46

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR LINTRCV for S32M24x. AUTOSAR LINTRCV driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR LINTRCV driver requirements and APIs are described in the AUTOSAR LINTRCV driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".

- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
C/CPP	C and C++ Source Code
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECUM	ECU state Manager
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
LIN	Local Interconnect Network
LINTRCV	Lin Transceiver Driver
SPI	Serial Peripheral Interface
ICU	Interrupt Control Unit
LSB	Least Significant Bit
MCU	Micro Controller Unit
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

3.2 Driver Design Summary

The LinTrcv Driver controls the LINPHY in AE of the S32M24x devices. It provides the following features:

- Compliant with LIN Physical Layer 2.2 specification.
- Configuration and initialization of the LINPHY.
- Support Normal and Standby modes.
- Support Wake Up Pattern (WUP) in Standby mode which covers 2 wakeup modes, internal wakeup by an upper layer or external wakeup.

3.3 Hardware Resources

There is one LIN Physical Layer (LINPHY) hardware channel on the Application Extension (AE) Subsystem. This is a companion die that is assembled along with MCU die, with which it communicates using mainly an SPI interface.

Note

The user must select properly the chip which supports LinPHY: P32M244LC, P32M242LC

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR LIN Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented, not fully implemented or out of scope for the LIN Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, not implemented or out of scope for the LINTRCV driver.

Requirement	Status	Description	Notes
SWS_LinTrcv_00099	N/S	The LIN bus transceiver driver might use drivers for Dio or Spi to control the LIN bus transceiver hardware. Thus these drivers must be available and ready to operate before the LIN bus transceiver driver is initialized.	This is not the case now for our driver, SPI is included via AE.
SWS_LinTrcv_00171	N/S	The LIN Transceiver Driver shall use the Time service Tm_Busy↔Wait1us16bit to realize the wait time for transceiver state changes.	TM module not being in scope for our product, The functionality is already covered by AE module and S↔PI module.
SWS_LinTrcv_00999	N/S	These requirements are not applicable to this specification.	This is not a requirement
ECUC_LinTrcv_00094	N/S	Container Name: LinTrcvDioAccess Parent Container: LinTrcvAccess Description: Container gives L↔IN transceiver driver information about accessing ports and port pins. In addition relation between LIN transceiver hardware pin names and Dio port access information is given. If a LIN transceiver hardware has no Dio interface, there is no instance of this container. Configuration Parameters	Not support to access DIO over M24x, M27x
ECUC_LinTrcv_00158	N/S	Container Name: LinTrcvDio↔ChannelAccess Parent Container↔: LinTrcvDioAccess Description↔: Container gives DIO channel access by single Lin transceiver channel. Configuration Parameters	Not support to access DIO over M24x, M27x

Requirement	Status	Description	Notes
ECUC_LinTrcv_00009	N/S	Name: LinTrcvHardwareInterface Name Parent Container: LinTrcv DioChannelAccess Description: LIN transceiver hardware interface name. It is typically the name of a pin. From a Dio point of view it is either a port, a single channel or a channel group. Depending on this fact either LINTRCV_DI↵O_PORT_SYMBOLIC_NAME or LINTRCV_DIO_CHAN↵NEL_SYMBOLIC_NAME or LINTRCV_DIO_CHANNEL↵_GROUP_SYMBOLIC_NAME shall reference a Dio configuration. The LIN transceiver driver implementation description shall list up this name for the appropriate LIN transceiver hardware.	Not support to access DIO over M24x, M27x
ECUC_LinTrcv_00102	N/S	Name: LinTrcvDioSymRefName Parent Container: LinTrcvDio↵ChannelAccess Description: Choice Reference to a DIO Port, DIO Channel or DIO Channel Group. This reference replaces the LINT↵RCV_DIO_PORT_SYM_NAME, LINTRCV_DIO_CHANNEL↵_SYM_NAME and LINTRC↵V_DIO_GROUP_SYM_NAME references in the Lin Trcv SWS.	Not support to access DIO over M24x, M27x
ECUC_LinTrcv_00159	N/S	Name: LinTrcvTimerType Parent Container: LinTrcv↵ General Description: Type of the Time Service Predefined Timer. Multiplicity: 0..1 Type: Ecuc↵EnumerationParamDef Range: None: None Timer_1us16bit: 16 bit 1us timer Post-Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time: X: All Variants	TM module not being in scope for our product, The functionality is already covered by AE module and S↵PI module
ECUC_LinTrcv_00160	N/S	Name: LinTrcvWaitTime Par↵ent Container: LinTrcvGeneral Description: Wait time for transceiver state changes in seconds.	Not apply for LinTrcv

Requirement	Status	Description	Notes
ECUC_LinTrcv_00155	N/S	Container Name: LinTrcvSpiSequence Parent Container: LinTrcvAccess Description: Container gives LIN transceiver driver information about one SPI sequence. One SPI sequence used by LIN transceiver driver is in exclusive use for it. No other driver is allowed to access this sequence. LIN transceiver driver may use one sequence to access n LIN transceiver hardware chips of the same type or n sequences are used to access one single LIN transceiver hardware chip. If a LIN transceiver hardware has no SPI interface, there is no instance of this container. Configuration Parameters	SPI is in AE module ownership.
ECUC_LinTrcv_00156	N/S	Name: LinTrcvSpiSequenceName Parent Container: LinTrcvSpiSequence Description: Reference to a Spi sequence configuration container.	SPI is in AE module ownership.

3.5 Driver Limitations

None

3.6 Driver usage and configuration tips

3.6.1 How to configure the SPI sequence via AE module

Ae module is used to access LIN transceiver hardware connected via SPI. This SPI sequence shalln't configure in LINTRCV module, It have to configure in Ae module.

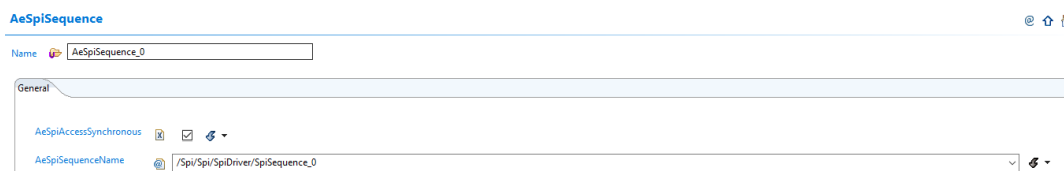


Figure 3.1 Reference to a Spi sequence configuration in Ae module over EB tresos.

SPI Configuration:

Configuration SPI sequence:

SpiSequence

Name  SpiSequence_0

General SpiJobAssignment

 SpiJobAssignment

Index	 SpiJobAssignment
0	 /Spi/Spi/SpiDriver/SpiJob_0

Figure 3.2 Configuration SPI sequence.

Configuration SPI Job:
In Spi Job tab, Need to configure the external device and assignment to Spi channel.

SpiJob

Name  SpiJob_0

General SpiChannelList

 SpiJobEndNotification  NULL_PTR

 SpiJobStartNotification  NULL_PTR

SpiJobId

 0  

SpiJobPriority

 0  

SpiDeviceAssignment

 /Spi/Spi/SpiDriver/SpiExternalDevice_0

Figure 3.3 Configuration SPI job.

In Spi External Device tab:

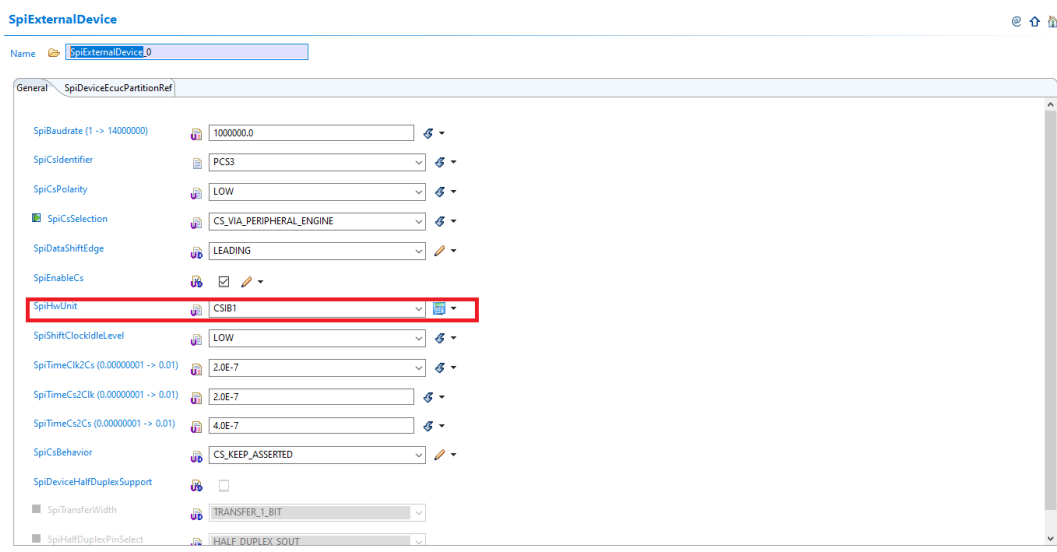


Figure 3.4 Configuration SPI External Device.

Note: The SpiHwUnit must refer to the Spi Phy Unit tab and the SpiPhyUnitMapping is LPSPI_1

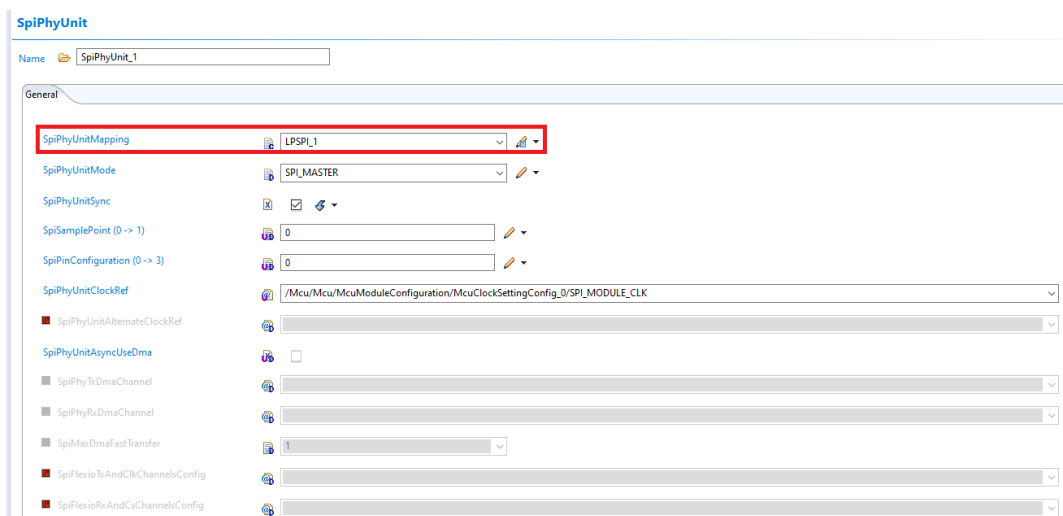


Figure 3.5 Configuration SPI channel.

In SpiChannelList tab:

Need to assign to the Spi Channel and configure for SpiChannel as below:

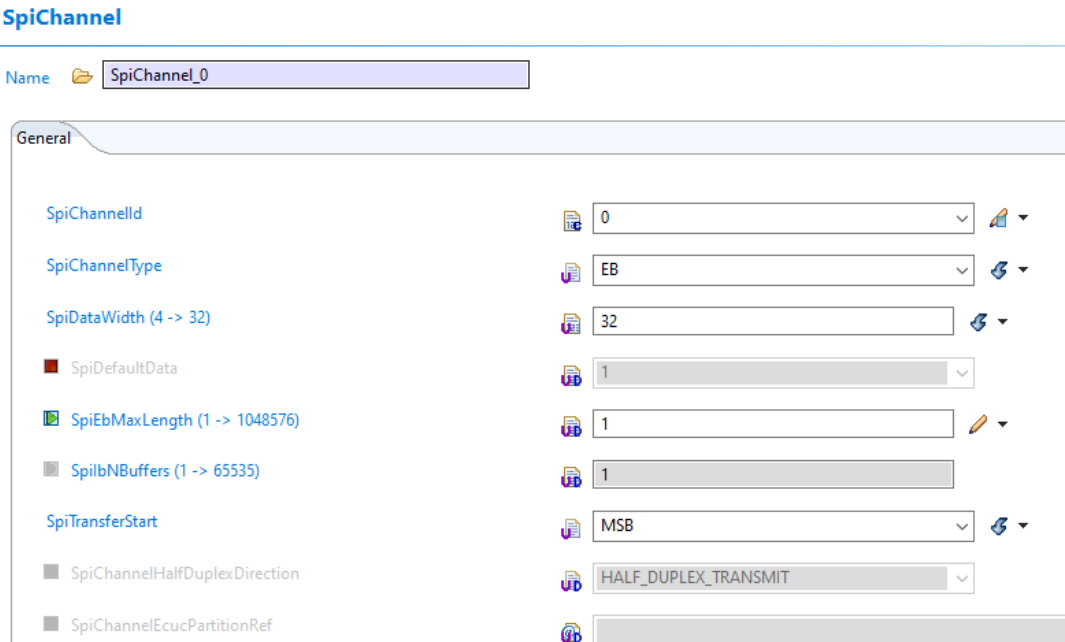


Figure 3.6 Configuration SPI channel.

Port Configuration:

Need to configure Pins for SPI in order communicates with LIN PHY interface.

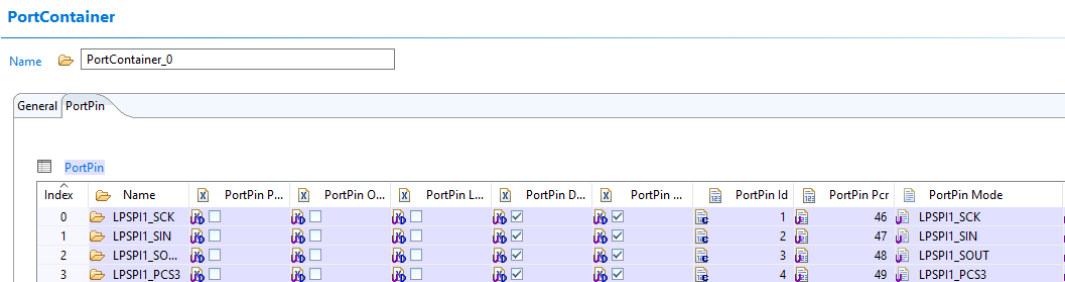


Figure 3.7 Configuration SPI Pins.

MCU Configuration:

Make sure that LPSP1_1 must enable clock.

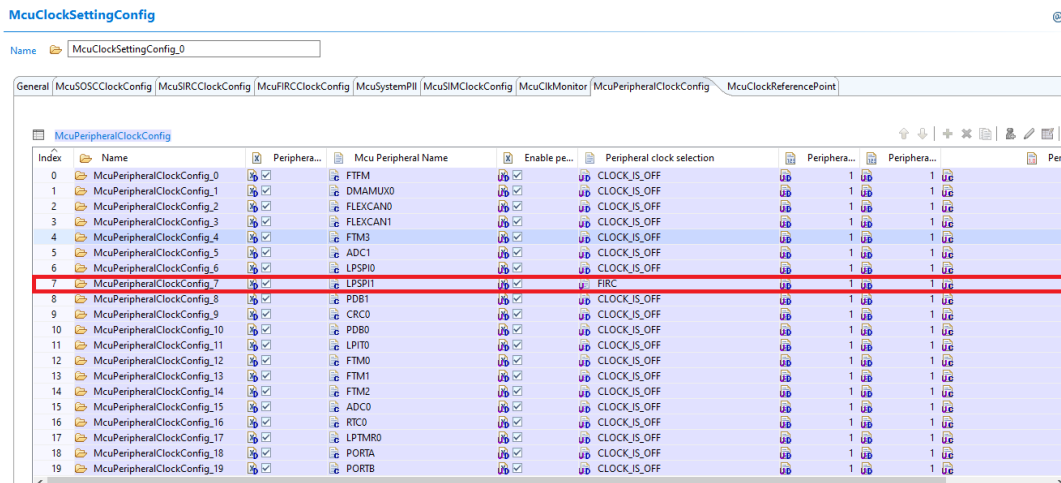


Figure 3.8 Enable the clock for LPSP1_1.

3.6.2 Wakeup notification handling

- The LinTrcv Driver supports the external wakeup signal detection.
- Dependency modules that must be configured:
 - ICU: Enable/disable the notification of wakeup event
 - PORT: Configure Icu channel for MCU which will capture signals on "INTERRUPT_OUT" pin sent from AEC
 - PLATFORM: Enable interrupt and install handler for "INTERRUPT_OUT" pin which is connected directly to PTD3 pin of MCU inside chip-package.
- In order to use LinTrcv wakeup feature, the steps below can be followed:

Step 1:
Enable wakeup feature on LinTrcv driver:

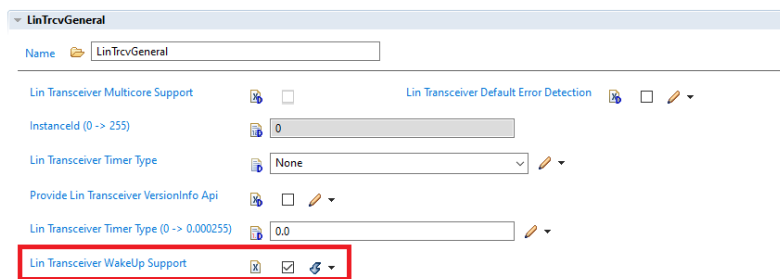


Figure 3.9 Enable Wakeup Feature.

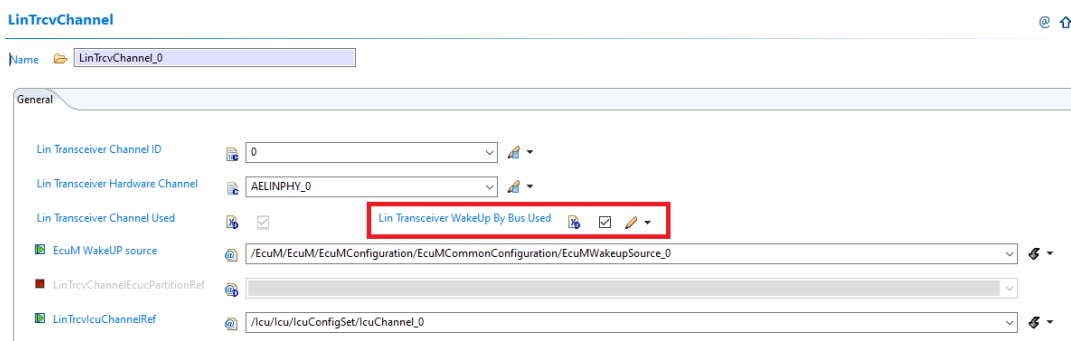


Figure 3.10 Enable Wakeup for individual Transceiver channel.

Step 2:

Configure Icu channel for MCU which will capture signals on "INTERRUPT_OUT" pin sent from AEC. The "INTERRUPT_OUT" pin is connected directly to PTD3 pin of MCU inside chip-package. The notifications can be events/faults.

The Icu channel must be configured for PTD3 pin.

IcuPort

Name  IcuPort_0

Port Modules IcuPortChannels

Port CI Hardware Module (dynamic range)  3 

Figure 3.11 IcuPort channel configuration.

IcuPort

Name  IcuPort_0

Port Modules IcuPortChannels

 IcuPortChannels

Index	Name	Port Cha...
0	IcuPortChannels_0	3

Figure 3.12 IcuPort channel configuration.

IcuSignalNotification must be enabled and referenced to "AEC_Common_IRQHandler" declared in AE driver.

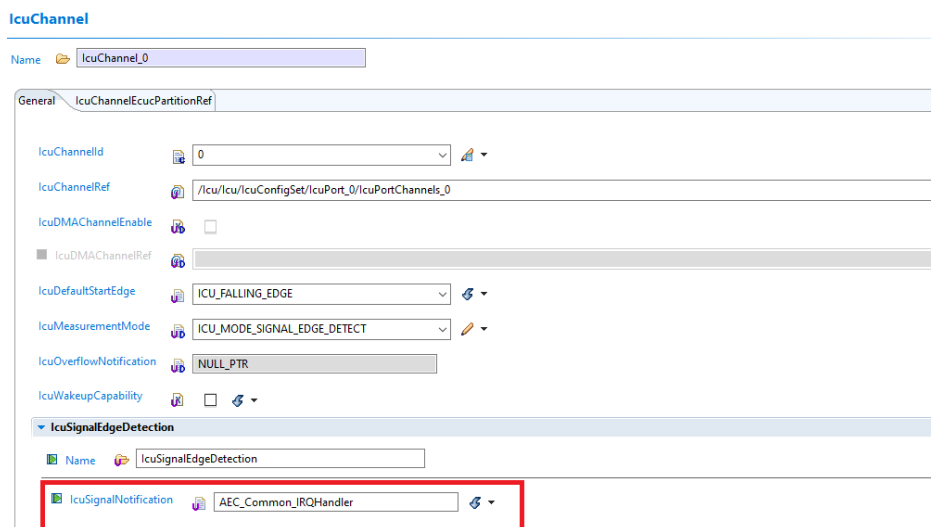


Figure 3.13 Icu channel configuration.

When the Icu notification is enabled and an AEC wakeup event is sent, the `"AEC_Common_IRQHandler"` function is invoked. This function of AE module will invoke the user's notification callback, which was defined in Step 4.

Note:

The PTD3 pin must be also configured in PORT driver

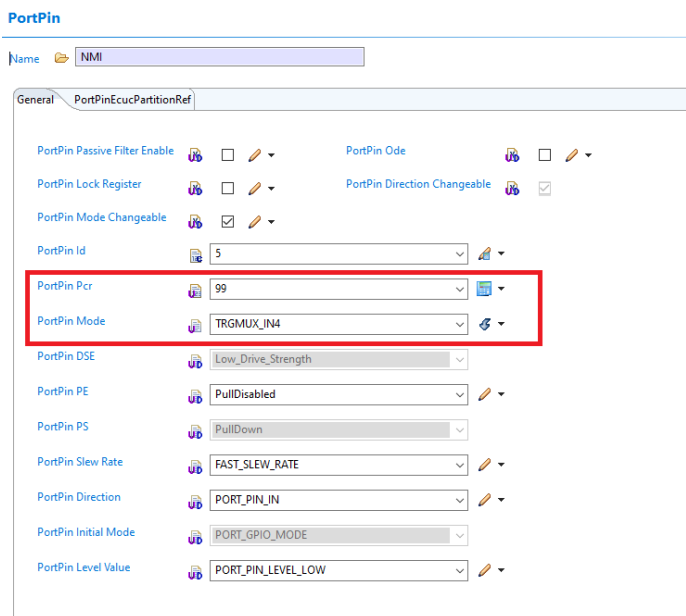


Figure 3.14 PTD3 configuration.

The interrupt for PTD3 pin must be configured in PLATFORM component everytime there are signals from AE Subsystem that will notify the main MCU.

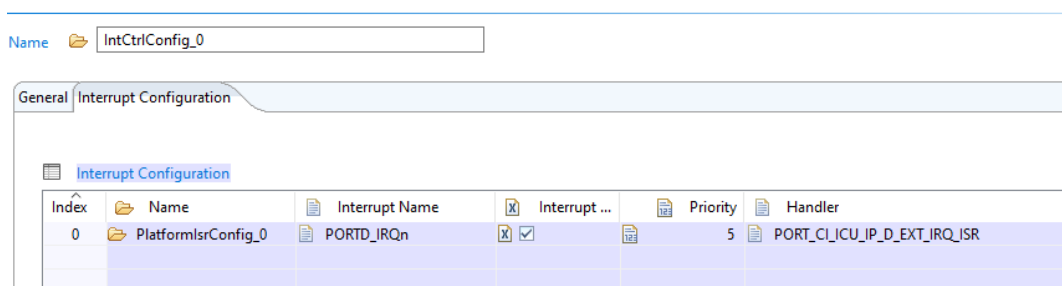


Figure 3.15 Interrupt for PTD3 pin.

Step 3:

Configures LinTrcvIcuChannelRef to reference to corresponding Icu channel configured at above step. so the enabling/disabling the Icu notification will be done in LinTrcv Driver.

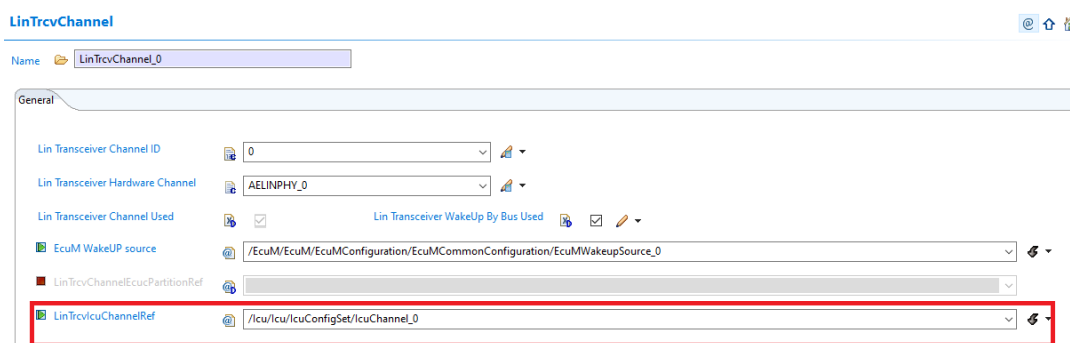


Figure 3.16 LinTrcvIcuChannelRef configuration.

Step 4:

Add user's notification callback which will be called after a wakeup notification sent from AEC.

Index	Name	EventorFaultName	AelrEnabled	AelrPriority	Handler
0	AelntConfig_0	LIN_INT	☒	0	EcuM_CheckWakeup
1	AelntConfig_1	OCD_VDDC	☐	1	undefined_handler
2	AelntConfig_2	TEMP_WDG_PMC	☐	2	undefined_handler
3	AelntConfig_3	TEMP_WDG_PHY	☐	3	undefined_handler
4	AelntConfig_4	CAN_INT	☐	4	undefined_handler
5	AelntConfig_5	LVD_VDDC	☐	5	undefined_handler
6	AelntConfig_6	WAKEUP	☐	6	undefined_handler
7	AelntConfig_7	HVI_ACTIVE	☐	7	undefined_handler
8	AelntConfig_8	HVI_AE_SUPPLY	☐	8	undefined_handler
9	AelntConfig_9	STATERESET	☐	9	undefined_handler
10	AelntConfig_10	FRAMEWIDTH	☐	10	undefined_handler
11	AelntConfig_11	GDU_FAULT_PROT	☐	11	undefined_handler
12	AelntConfig_12	GDU_INT	☐	12	undefined_handler
13	AelntConfig_13	DPGA_OC_NEG	☐	13	undefined_handler
14	AelntConfig_14	DPGA_OC_POS	☐	14	undefined_handler

Figure 3.17 AE notification.

___"LIN_INT"___ Event in AE component should be used only if the user wants to be notified when a Wakeup Pattern is detected by LINPHY. This callback function must call LinTrcv_43__AE_CheckWakeup to set a Wakeup event to EcuM module.

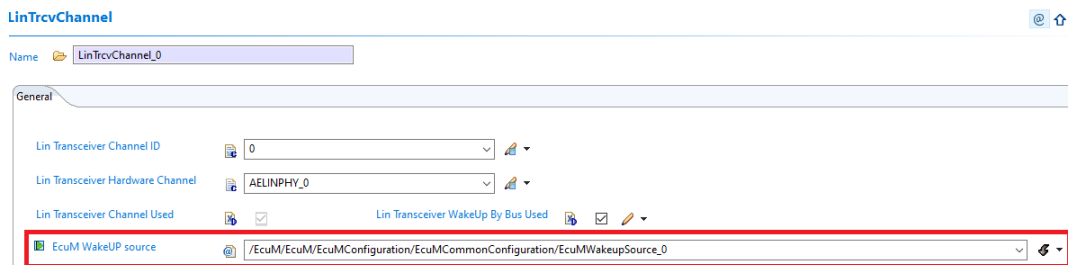


Figure 3.18 CanTrcvWakeupSourceRef configuration.

3.6.3 Approach about examples of LINTRCV

LinTrcv driver examples are available only for HLD driver. The LinTrcv examples are integrated with RTD LIN HLD driver. They can be found in the LIN driver plugin. The RTD supports a basic configuration for both EBT and Design Studio.

This example combines Lin with LinTrcv; instead of Lin requiring an external transceiver, LinTrcv is used, and a complete LIN frame (header + response) is delivered from Lin as the Master node.

3.7 Runtime errors

None

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

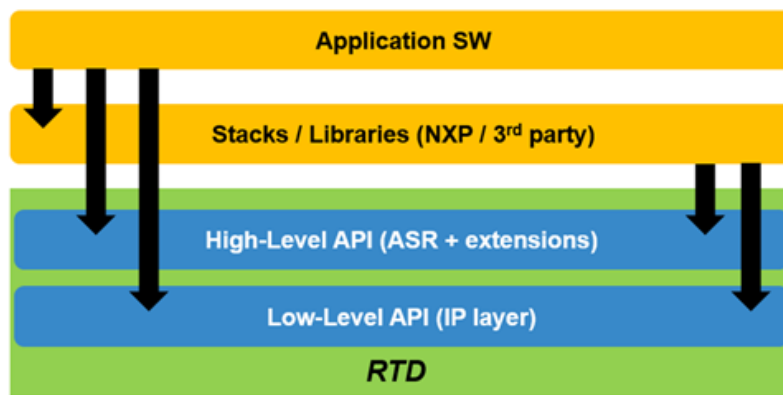


Figure 3.19 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world

3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
4. Initialization of specific drivers
5. Runtime functionality
6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

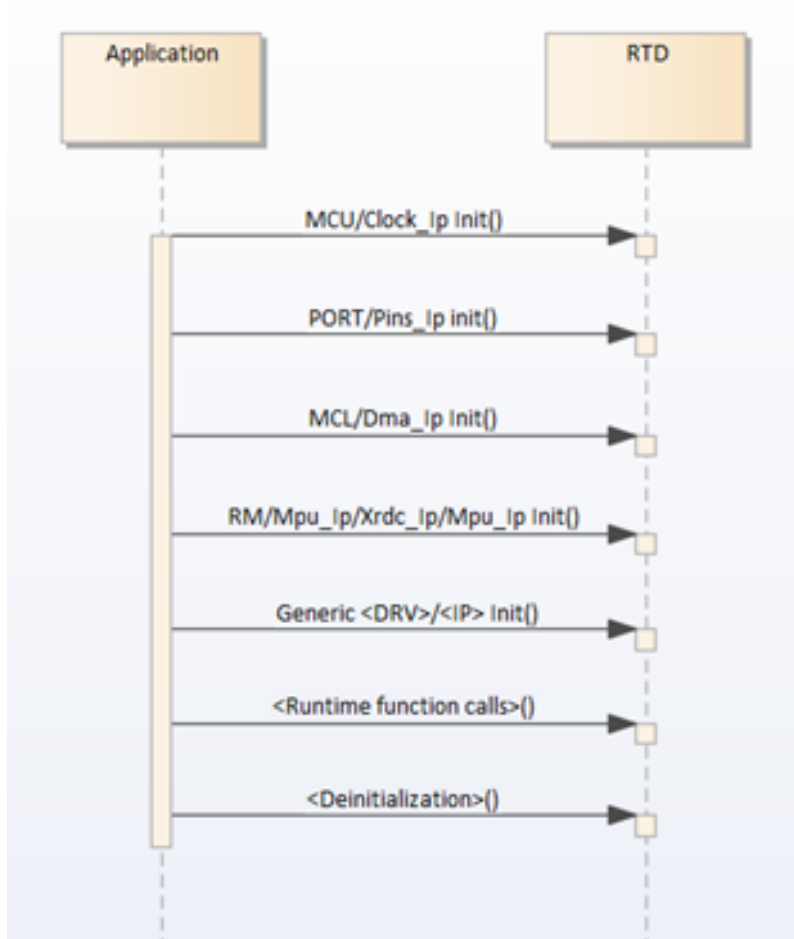


Figure 3.20 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

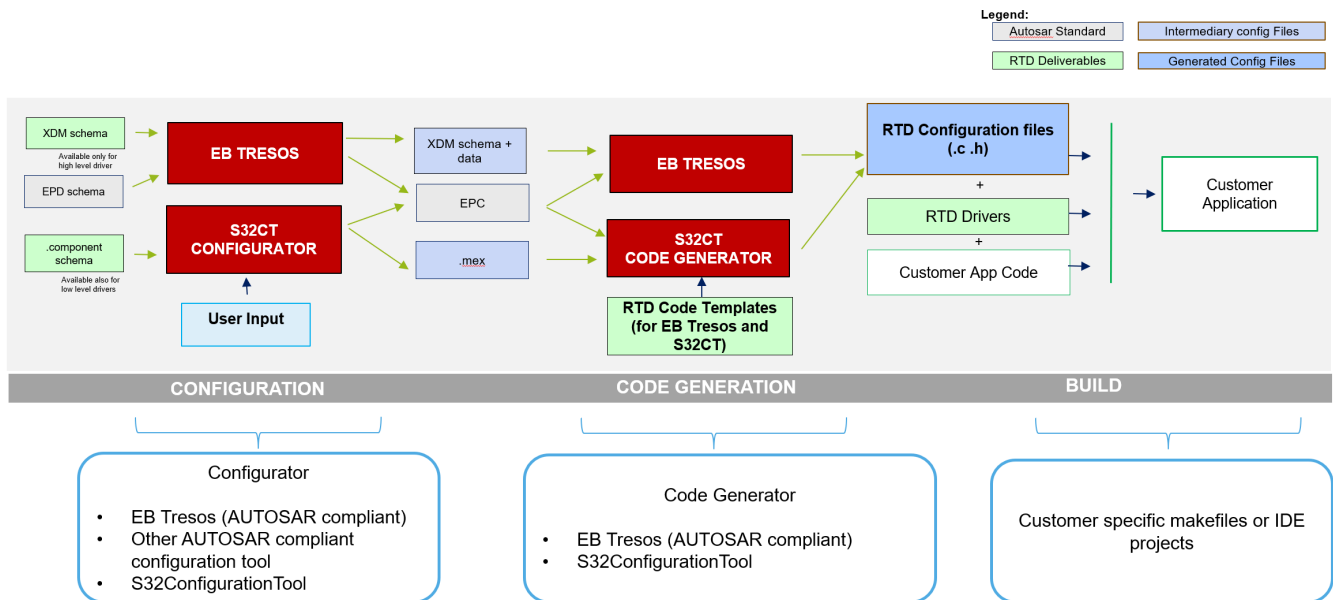


Figure 4.1 Configuration generation flow

- [LinTrcv](#)

4.1 LinTrcv

- Module [LinTrcv](#)

- Container [LinTrcvGeneral](#)
 - * Parameter [LinTrcvMultiPartitionSupport](#)
 - * Parameter [LinTrcvDevErrorDetect](#)
 - * Parameter [LinTrcvIndex](#)
 - * Parameter [LinTrcvTimerType](#)
 - * Parameter [LinTrcvVersionInfoApi](#)
 - * Parameter [LinTrcvWaitTime](#)
 - * Parameter [LinTrcvWakeUpSupport](#)
 - * Reference [LinTrcvEcucPartitionRef](#)
- Container [LinTrcvChannel](#)
 - * Parameter [LinTrcvChannelId](#)
 - * Parameter [LinTrcvHwChannel](#)
 - * Parameter [LinTrcvChannelUsed](#)
 - * Parameter [LinTrcvWakeupByBusUsed](#)
 - * Reference [LinTrcvWakeupSourceRef](#)
 - * Reference [LinTrcvChannelEcucPartitionRef](#)
 - * Reference [LinTrcvIcuChannelRef](#)
 - * Container [LinTrcvAccess](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module LinTrcv

Configuration of LIN Transceiver Driver module.

Included containers:

- [LinTrcvGeneral](#)
 - [LinTrcvChannel](#)
 - [CommonPublishedInformation](#)
-

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE

4.1.2 Container LinTrcvGeneral

LinTrcvGeneral

Autosar Requirements: ECUC_LinTrcv_00090

Container gives LIN transceiver driver basic information.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter LinTrcvMultiPartitionSupport

This parameter determine multi-Partition feature will be used in LinTrcv driver.

If LinTrcvMultiPartitionSupport is disabled, then for all the variants no partition shall be defined.

If LinTrcvMultiPartitionSupport is enabled, at least one EcucPartition needs to be defined (in all variants).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.4 Parameter LinTrcvDevErrorDetect

LinDevErrorDetect

Autosar Requirements: ECUC_LinTrcv_00001

Switches the Default Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.5 Parameter LinTrcvIndex

Autosar Requirements: ECUC_LinTrcv_00153

Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.

Note, this parameter is not used in the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.6 Parameter LinTrcvTimerType

LinTrcvTimerType

Type of the Time Service Predefined Timer.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	None
literals	['None', 'Timer_1us16bit']

4.1.7 Parameter LinTrcvVersionInfoApi

LinTrcvVersionInfoApi

Autosar Requirements: ECUC_LinTrcv_00003

Switches the LinTrcvVersionInfo function ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.8 Parameter LinTrcvWaitTime

LinTrcvWaitTime

Autosar Requirements: ECUC_LinTrcv_00160

Wait time for transceiver state changes in seconds.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Configuration Parameters

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.0
max	2.55E-4
min	0.0

4.1.9 Parameter LinTrcvWakeUpSupport

LinTrcvWakeUpSupport

Autosar Requirements: ECUC_LinTrcv_00107

Inform whether wake up is supported or not. In case wake up is not supported by LIN transceiver hardware the setting shall be false. The wake up ability may be switched on or off for each channel of one LIN transceiver by LinTrcvWakeUpSourceRef.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.10 Reference LinTrcvEcucPartitionRef

Maps the Lin transceiver driver to zero or multiple ECUC partitions to make the modules API available in this partition. The Lin transceiver driver will operate as an independent instance in each of the partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.11 Container LinTrcvChannel

Container gives LIN transceiver driver information about a single LIN transceiver channel. Any LIN transceiver driver has such LIN transceiver channels"

Included subcontainers:

- [LinTrcvAccess](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.12 Parameter LinTrcvChannelId

Unique identifier of the LIN Transceiver Channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	0

4.1.13 Parameter LinTrcvHwChannel

Selects the Lin Transceiver Hardware Channel.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Configuration Parameters

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	AELINPHY_0
literals	['AELINPHY_0']

4.1.14 Parameter LinTrcvChannelUsed

LinTrcvChannelUsed

Autosar Requirements: ECUC_LinTrcv_00004

Shall the related LIN transceiver channel be used?

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.15 Parameter LinTrcvWakeupByBusUsed

LinTrcvWakeupByBusUsed

Autosar Requirements: ECUC_LinTrcv_00006

Is wake up by bus supported? If LIN transceiver hardware does not support wake up by bus value is always FALSE. If LIN transceiver hardware supports wake up by bus value is TRUE or FALSE depending whether it is used or not

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.16 Reference LinTrcvWakeupSourceRef

LinTrcvWakeupByBusUsed

Autosar Requirements: ECUC_LinTrcv_00012

Reference to a wakeup source in the EcuM configuration. This reference is only needed if LinTrcvWakeupByBusUsed is true.

Implementation Type: reference to EcuM_WakeupSourceType

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuM/EcuMConfiguration/EcuMCommon← Configuration/EcuMWakeupSource

4.1.17 Reference LinTrcvChannelEcuPartitionRef

Maps one single Lin transceiver channel to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions where the Lin transceiver driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.18 Reference LinTrcvIcuChannelRef

Reference to the IcuChannel to enable/disable the interrupts for wakeups.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1

Configuration Parameters

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Icu/IcuConfigSet/IcuChannel

4.1.19 Container LinTrcvAccess

Container gives LIN transceiver driver access about a single LIN transceiver channel.

Included choices:

- LinTrcvDioAccess
- LinTrcvSpiSequence

Property	Value
type	ECUC-CHOICE-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.20 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.21 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.22 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.23 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0

Configuration Parameters

Property	Value
min	0

4.1.24 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	64
max	64
min	64

4.1.25 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.26 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.27 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.28 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

`<ModuleName>_<VendorId>_<VendorApiInfix>`.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	AE

4.1.29 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

AELINPHY_LINTRCV_IP	36
AeLinPhy LinTrcv IPL	38
LINTRCV Driver	40
LINTRCV Driver	46

Chapter 6

Module Documentation

6.1 AELINPHY_LINTRCV_IP

6.1.1 Detailed Description

Function Reference

- [AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_Init](#) (const uint8 Instance, const [AeLinPhy_LinTrcv_Ip_UserConfigType](#) *LinTrcvUserConfig)
- [AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_SetMode](#) (const uint8 Instance, const [AeLinPhy_LinTrcv_Ip_TransceiverModeType](#) OpMode)
- [AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_GetMode](#) (const uint8 Instance, [AeLinPhy_LinTrcv_Ip_TransceiverModeType](#) *TransceiverMode)
- [AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_SetWakeupMode](#) (const uint8 Instance, [AeLinPhy_LinTrcv_Ip_TrcvWakeupModeType](#) TrcvWakupMode)
- [AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_CheckWakeupFlag](#) (const uint8 Instance)

6.1.2 Function Reference

6.1.2.1 AeLinPhy_LinTrcv_Ip_Init()

```
AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_Init (
    const uint8 Instance,
    const AeLinPhy_LinTrcv_Ip_UserConfigType * LinTrcvUserConfig )
```

AeLinPhy_LinTrcv_Ip_Init

6.1.2.2 AeLinPhy_LinTrcv_Ip_SetMode()

```
AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_SetMode (
    const uint8 Instance,
    const AeLinPhy_LinTrcv_Ip_TransceiverModeType OpMode )
```

AeLinPhy_LinTrcv_Ip_SetMode

6.1.2.3 AeLinPhy_LinTrcv_Ip_GetMode()

```
AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_GetMode (
    const uint8 Instance,
    AeLinPhy_LinTrcv_Ip_TransceiverModeType * TransceiverMode )
```

AeLinPhy_LinTrcv_Ip_GetMode

6.1.2.4 AeLinPhy_LinTrcv_Ip_SetWakeupMode()

```
AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_SetWakeupMode (
    const uint8 Instance,
    AeLinPhy_LinTrcv_Ip_TrcvWakeupModeType TrcvWakupMode )
```

AeLinPhy_LinTrcv_Ip_SetWakeupMode

6.1.2.5 AeLinPhy_LinTrcv_Ip_CheckWakeupFlag()

```
AeLinPhy_LinTrcv_Ip_StatusType AeLinPhy_LinTrcv_Ip_CheckWakeupFlag (
    const uint8 Instance )
```

AeLinPhy_LinTrcv_Ip_CheckWakeupFlag

6.2 AeLinPhy LinTrcv IPL

6.2.1 Detailed Description

Data Structures

- struct [AeLinPhy_LinTrcv_Ip_UserConfigType](#)
User configuration structure of the LINTRCV driver. [More...](#)

Enum Reference

- enum [AeLinPhy_LinTrcv_Ip_TransceiverModeType](#)
Enum containing the events related to a mode of Lin Trcv.
- enum [AeLinPhy_LinTrcv_Ip_StatusType](#)
AELINPHY status type.
- enum [AeLinPhy_LinTrcv_Ip_TrcvWakeupModeType](#)
Enum containing the events related to a wakeup mode of Lin Trcv.

6.2.2 Data Structure Documentation

6.2.2.1 struct AeLinPhy_LinTrcv_Ip_UserConfigType

User configuration structure of the LINTRCV driver.

Definition at line 123 of file AeLinPhy_LinTrcv_Ip_Types.h.

Data Fields

uint8	Instance	Hardware Instance.
-------	----------	--------------------

6.2.3 Enum Reference

6.2.3.1 AeLinPhy_LinTrcv_Ip_TransceiverModeType

enum [AeLinPhy_LinTrcv_Ip_TransceiverModeType](#)

Enum containing the events related to a mode of Lin Trcv.

This enum defines types for an enumerating event related to mode of Lin Trcv.

Enumerator

AELINPHY_LINTRCV_IP_OFF_MODE	The LIN Physical Layer is fully disabled. No wake-up functionality is available.
AELINPHY_LINTRCV_IP_RECV_ONLY_MODE	Entering this mode disables the transmitter and immediately stops any on-going transmission, The receiver is running in full performance mode.
AELINPHY_LINTRCV_IP_NORMAL_MODE	The full functionality is available. Both receiver and transmitter are enabled.

Enumerator

AELINPHY_LINTRCV_IP_STANDBY_MODE	The transmitter of the LIN Physical Layer is disabled and the receiver enters a low power mode in order to be able to detect valid wake-up pulses on the LIN bus.
----------------------------------	---

Definition at line 82 of file AeLinPhy_LinTrcv_Ip_Types.h.

6.2.3.2 AeLinPhy_LinTrcv_Ip_StatusType

enum [AeLinPhy_LinTrcv_Ip_StatusType](#)

AELINPHY status type.

Enumerator

AELINPHY_LINTRCV_IP_STATUS_ERROR	No error. General error, command failed to execute task successfully.
----------------------------------	---

Definition at line 93 of file AeLinPhy_LinTrcv_Ip_Types.h.

6.2.3.3 AeLinPhy_LinTrcv_Ip_TrcvWakeupModeType

enum [AeLinPhy_LinTrcv_Ip_TrcvWakeupModeType](#)

Enum containing the events related to a wakeup mode of Lin Trcv.

This enum defines types for an enumerating event related to wakeup mode of Lin Trcv.

Enumerator

AELINPHY_LINTRCV_IP_WUMODE_ENABLE	The notification for wakeup events is enabled on the addressed network.
AELINPHY_LINTRCV_IP_WUMODE_DISABLE	The notification for wakeup events is disabled on the addressed network.
AELINPHY_LINTRCV_IP_WUMODE_CLEAR	A stored wakeup event is cleared on the addressed network.

Definition at line 107 of file AeLinPhy_LinTrcv_Ip_Types.h.

6.3 LINTRCV Driver

6.3.1 Detailed Description

Macros

- `#define LINTRCV_43_AE_NOT_ACTIVE`
LINTRCV driver states.
- `#define LINTRCV_43_AE_ACTIVE`
LINTRCV driver states.
- `#define LINTRCV_43_AE_SETOPMODE_ID`
API service ID for `LinTrcv_43_AE_SetOpMode()` function.
- `#define LINTRCV_43_AE_GETOPMODE_ID`
API service ID for `LinTrcv_43_AE_GetOpMode()` function.
- `#define LINTRCV_43_AE_GETBUSWUREASON_ID`
API service ID for `LinTrcv_43_AE_GetBusWuReason()` function.
- `#define LINTRCV_43_AE_CHECKWAKEUP_ID`
API service ID for `LinTrcv_43_AE_CheckWakeup()` function.
- `#define LINTRCV_43_AE_SETWAKEUPMODE_ID`
API service ID for `LinTrcv_43_AE_SetWakeupMode()` function.

Function Reference

- `void LinTrcv_43_AE_Init (const LinTrcv_43_AE_ConfigType *ConfigPtr)`
Initializes the LINTRCV module.
- `Std_ReturnType LinTrcv_43_AE_SetOpMode (uint8 LinNetwork, LinTrcv_TrcvModeType OpMode)`
The internal state of the LIN transceiver driver is switched to mode given in the parameter `OpMode`.
- `Std_ReturnType LinTrcv_43_AE_GetOpMode (uint8 LinNetwork, LinTrcv_TrcvModeType *OpMode)`
API detects the actual software state of LIN transceiver driver.
- `Std_ReturnType LinTrcv_43_AE_GetBusWuReason (uint8 LinNetwork, LinTrcv_TrcvWakeupReasonType *Reason)`
This API provides the reason for the wakeup that the LIN transceiver has detected in the parameter "Reason". The ability to detect and differentiate the possible wakeup reasons depends strongly on the LIN transceiver hardware.
- `Std_ReturnType LinTrcv_43_AE_CheckWakeup (uint8 LinNetwork)`
Notifies the calling function if a wakeup is detected.
- `Std_ReturnType LinTrcv_43_AE_SetWakeupMode (uint8 LinNetwork, LinTrcv_TrcvWakeupModeType TrcvWakupMode)`
This API enables, disables and clears the notification for wakeup events on the addressed network.

6.3.2 Macro Definition Documentation

6.3.2.1 LINTRCV_43_AE_NOT_ACTIVE

```
#define LINTRCV_43_AE_NOT_ACTIVE
```

LINTRCV driver states.

The state NOT_ACTIVE means that the LinTrcv module has not been initialized yet and cannot be used.

Definition at line 217 of file LinTrcv_43_AE.h.

6.3.2.2 LINTRCV_43_AE_ACTIVE

```
#define LINTRCV_43_AE_ACTIVE
```

LINTRCV driver states.

The ACTIVE state indicates that the LINTRCV driver has been initialized, making each available channel ready for service.

Definition at line 226 of file LinTrcv_43_AE.h.

6.3.2.3 LINTRCV_43_AE_SETOPMODE_ID

```
#define LINTRCV_43_AE_SETOPMODE_ID
```

API service ID for [LinTrcv_43_AE_SetOpMode\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 236 of file LinTrcv_43_AE.h.

6.3.2.4 LINTRCV_43_AE_GETOPMODE_ID

```
#define LINTRCV_43_AE_GETOPMODE_ID
```

API service ID for [LinTrcv_43_AE_GetOpMode\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 244 of file LinTrcv_43_AE.h.

6.3.2.5 LINTRCV_43_AE_GETBUSWUREASON_ID

```
#define LINTRCV_43_AE_GETBUSWUREASON_ID
```

API service ID for [LinTrcv_43_AE_GetBusWuReason\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 252 of file LinTrcv_43_AE.h.

6.3.2.6 LINTRCV_43_AE_CHECKWAKEUP_ID

#define LINTRCV_43_AE_CHECKWAKEUP_ID

API service ID for [LinTrcv_43_AE_CheckWakeup\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 261 of file LinTrcv_43_AE.h.

6.3.2.7 LINTRCV_43_AE_SETWAKEUPMODE_ID

#define LINTRCV_43_AE_SETWAKEUPMODE_ID

API service ID for [LinTrcv_43_AE_SetWakeupMode\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 269 of file LinTrcv_43_AE.h.

6.3.3 Function Reference

6.3.3.1 LinTrcv_43_AE_Init()

```
void LinTrcv_43_AE_Init (
    const LinTrcv_43_AE_ConfigType * ConfigPtr )
```

Initializes the LINTRCV module.

Parameters

in	<i>ConfigPtr</i>	- Pointer to the selected configuration set.
----	------------------	--

Returns

void

Precondition

-

6.3.3.2 LinTrcv_43_AE_SetOpMode()

```
Std_ReturnType LinTrcv_43_AE_SetOpMode (
    uint8 LinNetwork,
    LinTrcv_TrcvModeType OpMode )
```

The internal state of the LIN transceiver driver is switched to mode given in the parameter OpMode.

Parameters

in	<i>LinNetwork</i>	LIN network to wich API call has to be applied
in	<i>OpMode</i>	The parameter says to which operation mode the change shall be performed.

Returns

Std_ReturnType.

Return values

<i>E_OK</i>	will be returned if the transceiver state has been changed to the requested mode.
<i>E_NOT_OK</i>	will be returned if the transceiver state change is not accepted or has failed or the parameter is out of the allowed range.

6.3.3.3 LinTrcv_43_AE_GetOpMode()

```
Std_ReturnType LinTrcv_43_AE_GetOpMode (
    uint8 LinNetwork,
    LinTrcv_TrcvModeType * OpMode )
```

API detects the actual software state of LIN transceiver driver.

Parameters

in	<i>LinNetwork</i>	LIN network to which API call has to be applied
out	<i>OpMode</i>	Pointer to operation mode of the bus the API is applied to.

Returns

Std_ReturnType.

Return values

<i>E_OK</i>	will be returned if the operation mode is detected.
<i>E_NOT_OK</i>	will be returned, if service request is failed due to development errors or the operation mode is not detected.

6.3.3.4 LinTrcv_43_AE_GetBusWuReason()

```
Std_ReturnType LinTrcv_43_AE_GetBusWuReason (
    uint8 LinNetwork,
    LinTrcv_TrcvWakeupReasonType * Reason )
```

This API provides the reason for the wakeup that the LIN transceiver has detected in the parameter "Reason". The ability to detect and differentiate the possible wakeup reasons depends strongly on the LIN transceiver hardware.

Parameters

in	<i>LinNetwork</i>	LIN network to which API call has to be applied
out	<i>Reason</i>	Pointer to wakeup reason of the bus the API is applied to.

Returns

Std_ReturnType.

Return values

<i>E_OK</i>	will be returned if the wake up reason is detected.
<i>E_NOT_OK</i>	will be returned, if service request is failed due to development errors or the wakeup reason is not detected..

6.3.3.5 LinTrcv_43_AE_CheckWakeup()

```
Std_ReturnType LinTrcv_43_AE_CheckWakeup (
    uint8 LinNetwork )
```

Notifies the calling function if a wakeup is detected.

Parameters

in	<i>LinNetwork</i>	LIN network to which API call has to be applied.
----	-------------------	--

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	Will be returned, if no wakeup has been detected
<i>E_OK</i>	Will be returned, if a wakeup has been detected.

Precondition

: LinTrcv_43_AE_Init function must be called before this API.

6.3.3.6 LinTrcv_43_AE_SetWakeupMode()

```
Std_ReturnType LinTrcv_43_AE_SetWakeupMode (
    uint8 LinNetwork,
    LinTrcv_TrvcWakeupModeType TrcvWakupMode )
```

This API enables, disables and clears the notification for wakeup events on the addressed network.

Parameters

in	<i>LinNetwork</i>	LIN network to which API call has to be applied.
in	<i>TrcvWakupMode</i>	Requested transceiver wakup reason.

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	will be returned, if service request is failed due to development errors or the wakeup mode is not set
<i>E_OK</i>	will be returned if the transceiver state has been changed to the requested mode..

Precondition

: LinTrev_43_AE_Init function must be called before this API.

6.4 LINTRCV Driver

6.4.1 Detailed Description

Data Structures

- struct [LinTrcv_43_AE_ChannelConfigType](#)
LINTRCV channel configuration type structure. [More...](#)
- struct [LinTrcv_43_AE_ConfigType](#)
LINTRCV driver configuration type structure. [More...](#)

6.4.2 Data Structure Documentation

6.4.2.1 struct LinTrcv_43_AE_ChannelConfigType

LINTRCV channel configuration type structure.

This is the type of the external data structure containing the overall initialization data for one LINTRCV Channel. A pointer to such a structure is provided to the LINTRCV channel initialization routine for configuration of the LINTRCV hardware channel.

Definition at line 119 of file LinTrcv_43_AE_Types.h.

Data Fields

const uint8	LinTrcvChannelID	
const LinTrcv_43_AE_HwConfigType *	ChannelConfigPtr	Lin Channel ID. !<
const uint32	ChannelPartitionId	LINTRCV Hardware configuration pointer. !<
const boolean	AllocatedPartition	LINTRCV Channel partition id. !<
const boolean	LinTrcvChannelUsed	LINTRCV Channel is allocated partition or not. !<

6.4.2.2 struct LinTrcv_43_AE_ConfigType

LINTRCV driver configuration type structure.

This is the type of the pointer to the external data LINTRCV Channels. A pointer of such a structure is provided to the LINTRCV driver initialization routine for configuration of the LINTRCV hardware channel.

Definition at line 149 of file LinTrcv_43_AE_Types.h.

Data Fields

- const [LinTrcv_43_AE_ChannelConfigType](#) * [LinTrcv_43_AE_ChannelPtr](#) [LINTRCV_43_AE_HW_MAX_MODULES]
Partition id is assigned for this configuration.

Field Documentation

LinTrcv_43_AE_ChannelPtr

```
const LinTrcv\_43\_AE\_ChannelConfigType* LinTrcv_43_AE_ChannelPtr[LINTRCV_43_AE_HW_MAX_MODULES]
```


Partition id is assigned for this configuration.

!<

Hardware channel.

Constant pointer of the constant external data structure containing the overall initialization data for all the configured LINTRCV Channels.

Definition at line 158 of file LinTrcv_43_AE_Types.h.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.